

ELI — Orchestration And Agents

ELI v2.3.72

September 2026

Contents

ELI Orchestration & Agents — Full Topology	1
Two agent stacks (this is the key thing to understand)	1
Planning artifacts — 4 of them, only 1 drives execution	3
Where it is actually weak (corrected)	3
Recommended fixes (prioritized)	3
Update — 2026-06-09	4
Update — 2.3.7 (custom agents get a specification, and a real trust chain)	4

ELI Orchestration & Agents — Full Topology

Updated for v2.3.72. v2.3.37 eliminated the Quick-mode cliff: all CHAT modes run the orchestrator at scaled depth; retrieval is unified in `eli/memory/retrieval.py`; Stage 12 learning is centralized in `learning_coordinator.py`.

Supersedes the earlier `agent_bus.md`, which only documented the parallel specialist bus and missed the real `AgentOrchestrator`. Read-only reference; nothing here changes behaviour.

Source files: - `eli/cognition/orchestrator.py` — the real orchestrator (12-stage pipeline) - `eli/cognition/agent_bus.py` — the parallel 15-specialist bus - `eli/memory/retrieval.py` — shared turn retrieval (bus + orchestrator) - `eli/cognition/learning_coordinator.py` — Stage 12 `finalize_turn()` - `eli/kernel/pipeline_trace.py` — canonical S01-S12 logging - `eli/kernel/engine.py` — wiring / dispatch gate - `eli/execution/execution_planner.py` — declarative plan model (UNUSED) - `eli/planning/task_planner.py` — planner shim (stub)

Two agent stacks (this is the key thing to understand)

ELI has **one primary cognition path** for CHAT and a **parallel specialist bus** that the orchestrator **composes** (or that the engine falls back to):

1. `AgentOrchestrator` — the real orchestrator (`orchestrator.py`)

The 12-stage cognitive pipeline. The engine calls it for **every CHAT mode** (Quick through Expert) at a depth chosen by `mode_orchestrator_depth()` and `orchestrator_planner_mode()` (`reasoning_modes.py`). Components:

- **`PlannerAgent.plan_retrieval()`** (`orchestrator.py:53`) — produces a **mode-aware retrieval plan**:
 - fast: keyword only, no FAISS/RAG, KG only if identity, 1 ReAct iter, skip HyDE
 - balanced (default): keyword + semantic + KG, RAG if doc query, 3 ReAct iters
 - deep: everything, large budgets, full HyDE, 3 ReAct iters
- **`OrchestratorMemoryAgent`** (`orchestrator.py:131`) — delegates to **`retrieve_for_turn()`** in `eli/memory/retrieval.py` (shared with the bus). HyDE expansion → keyword/FTS5 + FAISS semantic + document RAG + KG

→ hybrid_merge → **heuristic rerank** (rerank_candidates). **Sequential** by design — the llama_cpp embedder is not thread-safe.

- **ExecutorAgent** (orchestrator.py:119) — thin wrapper over executor_enhanced.execute.

Flow inside AgentOrchestrator.run(): - **Non-CHAT actions** (orchestrator.py:539-740): dispatches the **AgentBus** for specialist evidence (line 561), then runs a **ReAct observation loop** (598-644): runs the executor, asks the loaded LLM ANSWER or TOOL:<action> <args>, and **chains to the next tool**, accumulating observations. 1 iter in fast mode, up to 3 otherwise. For “grounded synthesis” actions the observations are assembled into context and passed to the LLM; for direct actions the executor result is returned as-is. - **CHAT** (orchestrator.py:742-897): planner → shared retrieval → **dispatch_specialists()** (mode-aware fan-out; memory skipped when already prefetched) → context assembly → persona handoff → generation. Private reasoning modes (Normal/Advanced/Research/Expert) hand off to engine._run_chat_reasoning_loop. **The AgentBus is composed on the CHAT path** — not bypassed in Quick mode.

2. AgentBus — the parallel 15-specialist fan-out (agent_bus.py)

15 agents in _ALL_AGENTS (agent_bus.py:3071), each a _BaseAgent subclass with name + timeout_s:

#	name	timeout_s	accesses
1	memory	5.0	SQLite + FTS5 + FAISS; self-gates (_eli_memory_should_run)
2	system	8.0	direct action execution (SYSTEM_ACTIONS)
3	habit	3.0	user_patterns / habit tables
4	self_improvement	3.0	LoRA / self-tuning introspection
5	proactive	3.0	suggestion / anticipation
6	frontier	5.0	frontier / awareness model
7	plugin	6.0	plugin registry (PLUGIN_ACTIONS)
8	capability	6.0	capability manifest
9	voice	5.0	TTS / voice subsystem
10	orchestrator	3.0	bus-level planner — emits a plan dict only
11	file_code	4.0	source tree / code introspection
12	reflection	4.0	reflection log / insights
13	introspection	4.0	runtime / cognition self-inspection
14	knowledge_graph	3.0	entity / relation graph

Execution (AgentBus.dispatch, agent_bus.py:1533): - **Selective fan-out**: tiny filler chat → {memory, orchestrator}; non-chat → _select_agents_for_intent minimal set (keyword/action ladder, line 470); plain CHAT → broad fan-out across all _enabled agents. - **Parallel**: ThreadPoolExecutor, one thread per agent (or runtime_policy.budget("agent_workers", floor=4, ceiling=32)). - **Per-agent hard timeout**: future.result(timeout=agent.timeout_s) (1585); timeout/exception → failed AgentResult, never blocks the response. - **Aggregation** (_aggregate_confidence, 1248): per-agent contribution = evidence_quality × evidence_density × learned per-(agent,action) calibration; single-agent cap; empty-bus ceiling; corroboration bonus at ≥3 contributors. This part is genuinely solid.

When each runs (engine gate, v2.3.37+)

Situation	Path
CHAT (Quick / Normal / Advanced / Research / Expert)	AgentOrchestrator at mode depth → shared retrieval → dispatch_specialists()
Non-CHAT action (any mode)	AgentOrchestrator → bus + ReAct loop
Orchestrator returns None / raises	Falls back to AgentBus.dispatch() directly
Phatic / ultra-short filler (engine heuristic)	May use lean bus profile inside orchestrator

Stage 12 side effects (store turn, publish meta, `_learn_from_result`) run through `learning_coordinator.finalize_turn()` — one entry point for all CHAT exits.

Planning artifacts — 4 of them, only 1 drives execution

1. **ReAct loop** (`orchestrator.py:598`) — the *only* planner that actually sequences execution (tool → observe → decide → next tool).
2. **PlannerAgent.plan_retrieval** (`orchestrator.py:53`) — plans *retrieval* budgets, not actions. Used every CHAT turn.
3. **Bus OrchestratorAgent plan** (`agent_bus.py:1336`) — emits an `orchestrator_plan` dict stored in `trace["orchestrator_plan"]` (`engine.py:9026`) for display / persona handoff / status. **Never executed.**
4. **execution_planner.build_execution_plan** (`ExecutionPlan/PlanStep`) — now **wired in** as the canonical typed plan. `AgentBus.dispatch` builds it each turn, injecting the proven `_select_agents_for_intent` result as `agent_profile`, and drives active agents *through* `plan.agent_profile` (selection result unchanged). The plan is surfaced on `DispatchResult.execution_plan`. `EXECUTE_GOAL` also builds a real plan via it.
5. **task_planner.TaskPlanner** — no longer a hardcoded shim; it now delegates to `execution_planner.build_execution_plan` so there is a single real plan representation across the codebase.

Where it is actually weak (corrected)

1. **Retrieval unified (v2.3.37 — DONE).** `eli/memory/retrieval.py retrieve_for_turn()` is the single owner for semantic + conversation recall on a turn; 8 s turn cache prevents duplicate searches when orchestrator and bus both need evidence. FAISS deletes use **tombstones** (`mark_memory_deleted`) instead of silent stale vectors.
2. **The bus is still a single isolated round.** Within one dispatch, the 15 agents cannot consume each other's output. The ReAct loop chains *executor tool actions*, not bus agents — so a bus-level dependency (e.g. KG seeded by memory's entities) still cannot be expressed.
3. **Planning partially consolidated (improved).** `execution_planner` is now the canonical typed plan and drives bus selection; `task_planner` delegates to it. Remaining overlap: the engine's `_build_runtime_orchestrator_plan` (rich stage dict) and the bus `OrchestratorAgent plan` still co-exist with the typed `ExecutionPlan`. The ReAct loop remains the only planner that actually *executes* a sequence. A future pass could fold the stage dict into `ExecutionPlan.steps` so there is one plan type end-to-end.
4. **ReAct loop fragility — FIXED.** Now validates the proposed `TOOL:<action>` against `SUPPORTED_ACTIONS` (143 registered actions) and stops the loop on an unknown/hallucinated action instead of switching to it; merges `intent["args"]` (preserving originals) instead of overwriting. (`orchestrator.py ReAct block`.)
5. **Custom-agent timeout override — FIXED.** The override is now `_apply_runtime_policy_timeouts()`, applied to built-ins and **re-applied after `_load_custom_agents()`**, so custom agents get hardware-adapted timeouts too.
6. **Timeouts don't cancel work.** `future.result(timeout)` only stops waiting; a timed-out write-capable agent (memory/habit) can still land a late DB write.
7. **Confidence coupled to a fixed evidence-key schema.** `_evidence_density` only counts known keys; a custom agent returning useful prose under an unknown key contributes 0.0 to grounding.
8. **No early-exit for direct actions; failures debug-only.** Bus waits on the slowest selected agent even when a direct `action_result` is already in hand; timeouts/exceptions log at `log.debug` with no health surface (despite the `agent_metrics` table existing).

Recommended fixes (prioritized)

Priority	Fix	Why
Med	Move <code>_load_custom_agents()</code> above the runtime-policy timeout loop (or re-apply the override after loading).	One-line fix for the redistribution timeout gap (#5).
Med	Make the bus optionally two-round for dependent agents (round-1 results passed into round-2 <code>run()</code>), gated by a real plan.	Enables KG-seeded-by-memory etc. (#2).
Med	Either wire <code>execution_planner.ExecutionPlan</code> in as the bus's selection/sequencing source, or delete it + the shim <code>task_planner</code> to cut dead surface.	Resolves the 5-planner fragmentation (#3).
Med	Generic evidence-density floor: count any non-empty payload at low weight so unknown-key/custom agents aren't zeroed.	Makes custom agents first-class to confidence (#7).
Med	Cooperative-cancel token checked before write-capable agents commit.	Closes the late-write hole (#6).
Low	Early-return once a direct <code>action_result</code> exists; surface agent health (timeout rate, p95) from <code>agent_metrics</code> in <code>RUNTIME_STATUS</code> .	Latency + observability (#8).

Highest leverage now: **two-round bus composition (#2)** and **observability via pipeline_trace + agent health surfaces (#8)**.

Update — 2026-06-09

- **HabitAgent ↔ proactive disconnect fixed.** The bus `HabitAgent` used to read only `get_habit_rules()` (the usually-empty `habit_rules` table), so the chat path was blind to the behaviour the proactive daemon detects into the `habits` table. It now also reads `get_detected_habits()` and emits a summary + `detected_habits`; `AgentResult.has_evidence` recognises the new keys (it was judging the agent “no evidence” → dropped), and `DispatchResult.to_context_block` renders the habit summary into the chat context. Habit data is now consistent across the chat agent, `HABIT_STATUS`, and the persona overlay.
- **Goal autogenesis** (`planning/goal_autogenesis.py`) feeds the autonomy/goal-tick stack from ELI's own signals — see `runtime_planning_world.md`.

Update — 2.3.7 (custom agents get a specification, and a real trust chain)

The problem

A custom agent was a `.py` file dropped in a directory, `exec_module'd` at import, carrying a name, a `timeout_s` and an optional free-text “persona”. Nothing recorded what the agent was **for**, nothing defined **when**

it should fire, and nothing could tell whether it **worked**. An agent you cannot evaluate is an agent you cannot trust, improve or debug — it either seems fine or it does not.

Four concrete failures followed from that:

1. **No objective, prompt, triggers or measures.** “Persona” was the only steering available, and it was optional.
2. **Loaded from the installation.** The search paths were `eli/cognition/custom` and `eli/brain/agents/custom` — both inside the install tree, which is a read-only mount on a packaged build. An agent created through the GUI had nowhere valid to be saved.
3. **Load failures were invisible.** An untrusted or broken agent was skipped with a `log.debug` line, so the operator saw nothing at all.
4. **Registration was import-time only.** A newly created agent did nothing until ELI was restarted, with no message explaining why.

AgentSpec (`eli/cognition/agent_spec.py`)

An agent is now **data, not code**:

Field	Required	Purpose
objective	yes	one sentence on what this agent is responsible for
system_prompt	yes	the actual instruction the model receives
triggers	≥ 1	keyword · regex · action · always
success_criteria	≥ 1	runnable checks: contains, not_contains, regex, min/max length, non_empty, is_json
examples	no	input + expected checks, so the agent can be tested before going live
permissions	no	capabilities from the plugin vocabulary, gated at run time

`validate()` refuses vagueness rather than accepting it: an objective under 25 characters or matching a placeholder list (`todo`, `does stuff`, `helper`, ...) is rejected, as is a system prompt under 40 characters. An agent with no trigger is refused because it would register and silently never run; an agent with no success criterion is refused because nothing could tell whether it worked. An always trigger is allowed but warned about — it costs latency on every turn.

`evaluate(output)` runs the criteria and returns a score. That is the **measure** the wizard’s test step uses and the value `SpecAgent` reports as its confidence.

`content_hash()` deliberately excludes `created` and `enabled`, so re-saving a spec does not invalidate a trust grant while a real edit does.

SpecAgent (`agent_bus.py`)

Runs a spec: check triggers → check declared permissions → call the local model with the spec’s system prompt → score the output against the criteria. **An agent that fails its own success test contributes nothing** rather than polluting the bus with output nobody checked.

Because a spec executes no arbitrary code, spec agents need **no trust grant at all** — the whole hash/scan/provenance chain is only for the code agents that a prompt genuinely cannot cover.

Loader fixes

- `_custom_agent_dirs()` puts the **data dir first** (<data>/agents/custom), so a created agent has somewhere valid to live on any install.
- Trust is checked through `agent_trust.inspect()` (see `security.md` §5), which reports *why* an agent was not loaded.
- `agent_load_report()` records the per-agent outcome so the GUI can show the reason instead of it existing only as a debug line nobody reads.
- `reload_custom_agents()` re-scans specs and code **without a restart**, and *replaces* previously-registered spec agents rather than skipping them — skipping would mean an edited spec never takes effect, which is the same restart-required problem the function exists to remove.